

Regression Testing

Regression testing represents the re-execution of some subsets of tests that have already been conducted to ensure that changes have not propagated unintended side effects.

It is very useful in verifying the correctness of the code base after the underlying bugs have been detected and corrected.

Our regression strategy was to run all test cases, identify the failed ones, find the errors, correct them and re-execute the failed test cases in order to verify that the corrections to the code provided correct results.

In our assignment, we implemented tests for the `PaymentProcessor` class in which we found a bug in the `processPayment` method. All combinations of discount scenarios were covered to ensure correct tax application. When running the test cases: 6 out of 11 tests failed: `testCreditcardFirstOrder`, `testPaypalNonFirstOrder`, `testCashNonFirstOrder`, `testCreditCardNonFirstOrder`, `testPaypalFirstOrder` and `testCashFirstOrder` for the original code. For example, by running `testCreditCardFirstOrder` the expected amount was 97.75, but the received amount was 85.

From this, we deduced that originally, the 15% tax was applied before the discount, which was incorrect. We changed the code section :

```
// BUG: the tax of 0.15 should be applied AFTER the discount!
double taxedAmount = amount * (1 + 0.15); // aplying a tax of 0.15

double discountedAmount = amount * (1 - discount);

double finalAmount = discountedAmount;
```

in the `process payment` method of the `PaymentProcessor` class with:

```
double discountedAmount = amount * (1 - discount);
double finalAmount = discountedAmount * (1 + 0.15);
```

After which, we re-ran the test cases that failed and obtained 100% results, all tests passing. We selected the 6 tests related to `processPayment()` because the logic inside that method was changed.

Justification:

We used the Risk-Based Regression Testing and Selective Retest Based on Code Impact approaches:

1. Only methods impacted by the code change were considered (`processPayment()`).
2. Tests targeting unmodified logic were skipped.
3. All combinations of discount scenarios were covered to ensure correct tax application.

This approach is efficient because it reduces testing time and cost and maintains coverage where bugs are most likely to appear.